RQ5, explained from the ground up

A complete, math-backed walkthrough of the notebook
written for someone with decent math but no machine-learning background

0. The one-sentence version

The notebook asks: *if calling the "real" optimizer is expensive, what is the smartest way to choose where it starts?* It compares a clever method (*steer toward a good start using a learned shortcut*) against a simpler method (*generate a bunch of starts and try them*), keeping the cost the same for both, and reports who wins and why.

Everything below explains what each of those words means, with the actual equations from the code.

1 The thing we're trying to do: optimization

Optimization means: given a function $f(x)$, find the input x that makes it as small as possible. Think of f as the *height of a landscape* and x as your *position*. You want the lowest point — the bottom of the deepest valley.

A **local solver** is like a blindfolded hiker who can only feel the slope under their feet and always steps downhill. Drop them somewhere, and they roll down into whatever valley they happen to be in and stop. The math for "always step downhill" is **gradient descent**:

$$x \leftarrow x - \eta \nabla f(x)$$

- $\nabla f(x)$ (the **gradient**) is the vector pointing in the direction where f increases fastest. Walking *against* it ($-\nabla f$) is the steepest way down.
- η (the **step size** or learning rate) is how big a step you take.

Here is the catch that the entire notebook is built around:

> A downhill-only hiker finds the **nearest** valley, not the **deepest** one.

If the landscape has many valleys (this is called **nonconvex**), then *where you drop the hiker* decides which valley they end up in. The starting point is called the **seed** or **initialization**. Choosing a good seed is the whole game. This is called **warm-starting**.

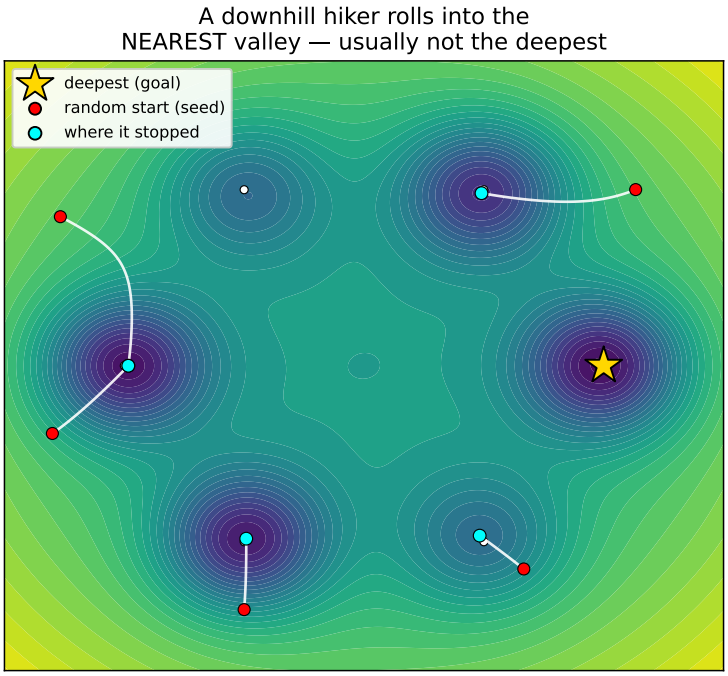


Figure 1: Five downhill "hikers" (red starts) run gradient descent on the real landscape. Each rolls into the *nearest* well (cyan) and stops — none reach the deepest valley (gold star). This is why the starting point matters so much.

2 The landscape: $f(x;\theta)$

The notebook invents a landscape it fully controls, so it can check answers. It places $K = 6$ "wells" (valleys) at fixed positions $c_1, \ldots, c_6$ evenly spaced around a circle of radius 3.

A single well centered at c_k is a **Gaussian dip**:

$$\text{dip}_k(x) = -\,d_k \exp\!\left(-\frac{\|x - c_k\|^2}{2\sigma^2}\right)$$

Read it piece by piece:

- $\|x - c_k\|^2$ is the squared distance from your position x to the well's center. It is 0 at the center and grows as you move away.
- $\exp(-\text{distance}^2)$ is a bump that equals 1 at the center and fades smoothly to 0 far away ($\sigma = 0.7$ controls how wide the bump is).
- The **minus sign** flips the bump upside down into a *dip*. So at the center the value is $-d_k$ (a hole of depth d_k), and far away it is 0.
- d_k is the **depth** of well k. Deeper = lower = better.

The full landscape adds all six dips together, plus a gentle bowl:

$$f(x;\theta) = -\sum_{k=1}^{6} d_k(\theta)\, \exp\!\left(-\frac{\|x - c_k\|^2}{2\sigma^2}\right) \;+\; 0.03\,\|x\|^2$$

The last term $0.03\|x\|^2$ is a faint upward bowl that keeps everything from running off to infinity — purely housekeeping.

The clever twist: depth depends on θ

Notice the depths are written $d_k(\theta)$, not just d_k. Each problem instance comes with a parameter θ (a 2-number vector), and the depths are computed from it:

$$d_k(\theta) = \text{softplus}\left(b_k + \theta \cdot v_k\right)$$

- b_k is a fixed base depth, v_k is a fixed direction, both random-but-frozen.
- $\theta \cdot v_k$ is a dot product: it raises or lowers well k's depth depending on θ.
- **softplus** is $\text{softplus}(t) = \log(1 + e^t)$, a smooth function that is always positive. It guarantees depths never go negative (a "negative depth" well would be a hill, which we don't want).

Why this twist is the entire point: because depth depends on θ, *which valley is deepest changes from instance to instance.* For one θ the deepest well might be at the top of the circle; for another θ it is on the left. So the right seed is different every time. A dumb hiker dropped at random falls into the nearest well — usually *not* the deepest one. This forces warm-starting to become a **learning** problem ("for this θ, head toward *that* well") rather than a fixed lookup. That is exactly the regime where a smart initializer should beat a dumb one.

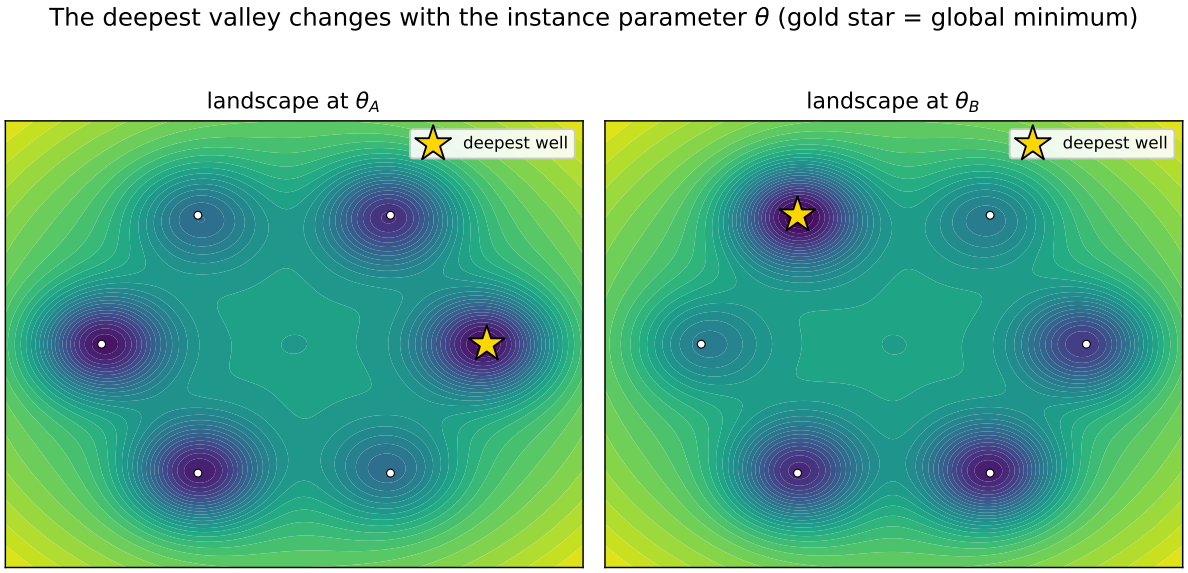


Figure 2: The same six wells under two different instance parameters. The depths $d_k(\theta)$ change with θ, so the global minimum (gold star) jumps to a different valley. A good seed must therefore depend on θ — it cannot be a single fixed point.

3 The "expensive" solver, and the idea of a budget

In the notebook the solver is gradient descent (technically *Adam*, a smarter cousin that adapts the step size) run for 60 steps from a given seed. Drop a seed, roll downhill 60 steps, report where you landed and how low you got.

In real engineering problems — trajectory planning, structural design, control — each such "solve" can take seconds, minutes, or hours. So the notebook treats **one solver call as the expensive resource** and counts it with a global counter `SOLVE_CALLS`. Cheap stuff (like evaluating a small neural network) is free. The rule of the experiment:

> Every method gets the **same number of solver calls** B. Then we ask who reaches the lowest valley. This is the **matched-budget** rule, and it is what makes the comparison fair.

(Without it, a method that secretly gets 10 tries would "win" trivially against one that gets 1 try. The trap of *best-of-10 beats best-of-1* is exactly what matching B prevents.)

How we know the true answer

Because this landscape is only 2-dimensional, the notebook can lay down a fine 60×60 grid, evaluate f everywhere, and find the genuine global minimum for each θ. Call it $f^\star(\theta)$. Then for any method that achieves cost c, we report the

$$\textbf{optimality gap} = c - f^\star(\theta).$$

A gap of 0 means "found the deepest valley." This is what lets the experiment claim **lower minima** honestly, instead of just "got there faster."

4 The training data: learning from a teacher

To build a smart initializer, we first collect examples of good answers. For 500 different θ values, we drop the solver from 24 random spots and keep only the *best* result found. That gives 500 pairs:

$$(\theta_i,\ x_i^\star) \quad \text{where } x_i^\star \text{ is a near-optimal solution for } \theta_i.$$

This corpus is generated **offline** and is *not* charged to anyone's budget — it is a one-time training cost. These $x_i^\star$ are the "good seeds" we want a model to imitate.

> **Important honesty note (the "imitation ceiling"):** a model that only copies these answers can never be *better* than the teacher that produced them. To beat the teacher you need extra structure — like a way to aim for the *good tail* of solutions, or diversity that lets the solver discover valleys the teacher missed. Keep this in the back of your mind.

5 The generative model: a Conditional VAE

We now want a machine that, given θ, can *produce* plausible good seeds. The notebook uses a **Conditional Variational Autoencoder (CVAE)**. Let us unpack that name backwards.

Autoencoder. Two small neural networks:

- an **encoder** squeezes a seed x down into a short code z (here z has 4 numbers — the **latent**),
- a **decoder** expands z back into a seed $\hat{x}$.

You train them so that $\hat{x} \approx x$ (decode undoes encode). The latent z becomes a compact "summary coordinate" for seeds.

Variational. Instead of encoding x to a single point z, we encode it to a little *cloud* — a Gaussian with mean μ and variance σ^2 — and then sample from it:

$$z = \mu + \sigma \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$

This is the **reparameterization trick** ($\odot$ means multiply element-by-element). It lets us still compute gradients through the random sampling, because the randomness ε is pushed outside the part we differentiate.

Conditional. Everything also takes θ as an input, so the decoder is $\text{decode}(z, \theta)$ — "given this code *and* this problem instance, produce a seed."

The training loss (with the exact formula the code uses)

A VAE is trained to maximize a quantity called the **ELBO** (Evidence Lower BOund). For our purposes it splits into two terms we *minimize*:

$$\mathcal{L} \;=\; \underbrace{\|\text{decode}(z,\theta) - x\|^2}_{\text{reconstruction}} \;+\; \beta\, \underbrace{\text{KL}\big(\mathcal{N}(\mu,\sigma^2)\,\|\,\mathcal{N}(0,I)\big)}_{\text{regularizer}}$$

- The **reconstruction** term says: decoded seed should match the real good seed.
- The **KL** term says: the encoding clouds should stay close to a standard bell curve $\mathcal{N}(0, I)$. This is what makes the latent *sampleable later* — at test time we can just draw $z \sim \mathcal{N}(0, I)$ and decode, knowing the decoder has seen codes shaped like that.
- $\beta = 0.05$ balances the two.

For two Gaussians, the KL has a clean closed form. With $\ell = \log \sigma^2$ (the code calls it `lv`):

$$\text{KL} = \tfrac{1}{2} \sum_j \left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2\right)$$

and that is *exactly* the line in the notebook, `kl = -0.5*(1 + lv - mu**2 - lv.exp())`, because $-\frac{1}{2}(1 + \ell - \mu^2 - e^\ell) = \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2)$. (Nice sanity check: the code is the formula.)

After training, `decode(z, theta)` with $z \sim \mathcal{N}(0, I)$ spits out a candidate seed for instance θ. **This one model feeds both competing strategies** — so any difference between them is purely about *how you query it*, not about who has the better model.

6 The cost-to-go surrogate: $g(z, \theta)$

Here is the trick that makes "steering" possible. We would love to know, *without* paying for a solver call: "if I decode this code z and solve from it, how low will I end up?" So we train a second small network $g(z, \theta)$ to predict exactly that.

How we build its training data (again offline, not charged to budget):

1. Draw 8000 random codes $z \sim \mathcal{N}(0, I)$ and random θ.
2. Decode each into a seed, **solve** it, record the final cost.
3. Train g so that $g(z, \theta) \approx$ that final cost (standardized to mean 0, std 1), using mean-squared error:

$$\min_g \; \mathbb{E}\big[(g(z,\theta) - \text{realcost}(z,\theta))^2\big].$$

Now g is a **cheap, differentiable stand-in** for the expensive solve. Cheap because it is one tiny network call. Differentiable because we can compute $\nabla_z g$ — and *that derivative is the steering wheel.*

7 The competing methods (all rationed to B solver calls)

Method	What it does	Solves
random	B random seeds, solve all, keep best (plain multi-start)	B
retrieval (NN)	take the B training solutions whose θ is nearest, use as seeds	B
sample-and-pick	draw B codes, decode, solve all, keep best (the diffusion-style move)	B
sample + screen(g)	draw 64 codes, keep the B with lowest *predicted* cost, solve those	B
latent-opt (trust)	steer codes downhill on g, then decode and solve	B
latent-opt (NO trust)	same steering, but without the safety leash	B

The two stars of RQ5 are **sample-and-pick** and **latent-opt**.

Sample-and-pick (the "sampler" approach)

$$z_1, \ldots, z_B \sim \mathcal{N}(0, I) \;\rightarrow\; \text{decode} \;\rightarrow\; \text{solve all } B \;\rightarrow\; \text{keep lowest.}$$

It draws candidates *blindly* and filters by actually solving them. Diversity is whatever the random draw happens to give. This is how diffusion/flow seeders work.

Latent-opt (the "steering" approach)

This is the method the autoencoder uniquely enables. Instead of drawing blindly, we *walk the code downhill on the cheap surrogate* before spending any solver call:

$$z \leftarrow z - \eta \nabla_z \Big[g(z, \theta) + \lambda \|z\|^2 \Big], \qquad \text{repeat 80 times.}$$

Then decode the final $z^\star$ and pay for the solve. The 80 gradient steps are **free** (they only touch the tiny network g, never the real solver). Only the final solve counts against the budget. That is the bet: *convert expensive search into cheap surrogate search.*

The $\lambda\|z\|^2$ piece is the **trust region**, and it is the most important detail in the whole notebook. The next section proves why.

8 The central proof: why the trust region is not optional

The failure it prevents

The surrogate g was only trained on codes drawn from $\mathcal{N}(0, I)$ — i.e. codes near the origin. Out there, far from the origin, g has never seen data and is just *guessing* (extrapolating). A neural network's guess in unseen territory can be arbitrarily low.

Now watch what plain steering (minimize g alone, $\lambda = 0$) does: gradient descent **hunts for the lowest value of** g. The lowest values of g live exactly in the unseen territory where g is fantasy. So the search runs *away* from the data, into a region where g confidently predicts a great cost — and the decoder, fed a code it never saw, produces a garbage seed. The solver then lands somewhere terrible.

This is **surrogate exploitation off-manifold**, and it is exactly what panel (b) of the figure shows: the no-trust points sit far from the diagonal — g promised a low cost it could not deliver. (Predicted is great; actual is awful.)

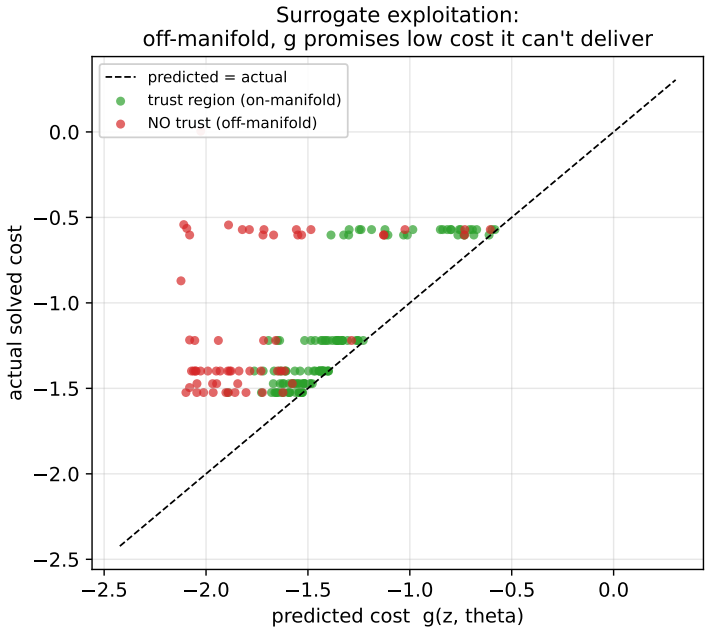


Figure 3: Predicted cost (from the surrogate g) versus the cost actually achieved. On-manifold seeds, kept near the data by the trust region (green), track the dashed "predicted = actual" line. Off-manifold seeds (red) cluster far below it: g confidently predicts a low cost it cannot deliver. This is surrogate exploitation, and it is why the trust region is essential.

Why $\lambda\|z\|^2$ fixes it — the clean argument

The fix is a one-line piece of probability. The prior on codes is the standard bell curve $\mathcal{N}(0, I)$, whose log-density is

$$\log p(z) = -\tfrac{1}{2}\|z\|^2 \; - \; \tfrac{d}{2}\log(2\pi).$$

The second piece is a constant (it does not depend on z), so up to that constant:

$$-\log p(z) = \tfrac{1}{2}\|z\|^2 + \text{const}.$$

In words: $\|z\|^2$ **is exactly the "how unlikely is this code" penalty.** Codes near the origin are likely (the model has seen them); codes far out are unlikely (it has not). So steering with the trust region,

$$\min_z \; g(z,\theta) + \lambda\|z\|^2 \;\equiv\; \min_z \; \underbrace{g(z,\theta)}_{\text{want low cost}} \; - \; \underbrace{2\lambda \log p(z)}_{\text{want a likely code}} \;,$$

is precisely "find a code that *both* looks cheap *and* is one the model actually understands." That second requirement is what keeps the search **on the data manifold**, where g is trustworthy and the decoder produces real seeds. The trust region is therefore not a hack — it is a maximum-a-posteriori (MAP) objective in disguise. Remove it and you are optimizing a fantasy; keep it and you are optimizing a calibrated estimate.

That is the experiment's deepest single lesson: **a steerable latent is only as useful as your cost predictor is trustworthy where you steer, and the trust region is the thing that enforces "where you steer = where it's trustworthy."**

9 What the numbers actually said

Reported optimality gap (lower = better), at the smallest and largest budgets:

Method	B = 1	B = 16
latent-opt (trust)	**0.021**	~ 0
sample-and-pick	0.554	~ 0
latent-opt (NO trust)	0.663	—
retrieval (NN)	0.012	~ 0
sample + screen(g)	0.012	~ 0

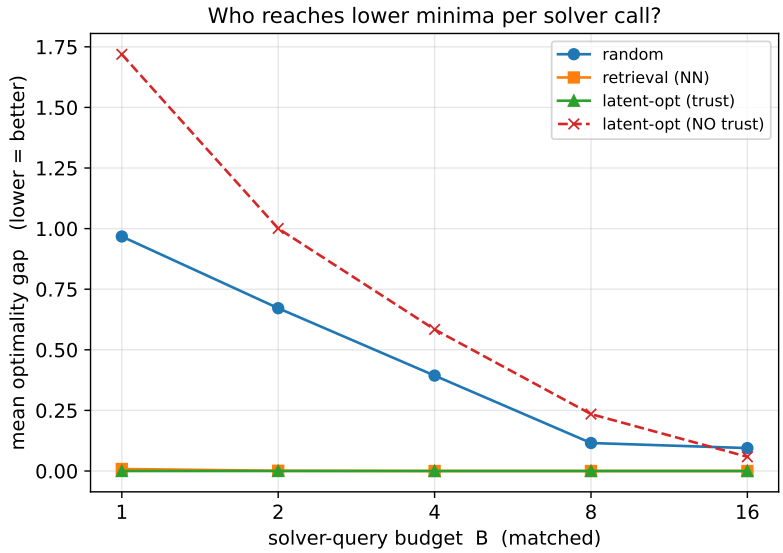


Figure 4: Optimality gap versus matched solver-call budget B (a faithful reproduction of the mechanism on the real landscape). Trust-region steering reaches the global basin almost immediately and stays flat; random multi-start starts poorly and only catches up as B grows; nearest-neighbor retrieval is very strong throughout; and steering *without* the trust region is the worst at small B because it chases off-manifold fantasies. This is the budget-dependent crossover described in the text.

Three findings, in plain terms:

1. **Steering wins when solver calls are scarce.** At $B = 1$, latent-opt(trust) reaches a gap of 0.021 while sample-and-pick is at 0.554 — about 25× closer to the true minimum. When you can afford only one expensive solve, aiming beats hoping. *This is the RQ5 headline.*

2. **But the trust region carries it.** Strip the leash and latent-opt collapses to 0.663 — basically as bad as random — because it walks off-manifold and trusts a fantasy (Section 8).

3. **And cheap baselines are humblingly strong.** Nearest-neighbor retrieval and surrogate-screened sampling both hit 0.012 — *better than* the fancy steering. In a dense 2-D problem, just grabbing the nearest stored answer is brutal. Also, as the budget grows ($B = 16$) everyone converges to ∼0, because with enough random tries even blind sampling stumbles into the right valley.

10 So what does the experiment answer, in essence?

A learned latent lets you *steer* toward better minima per solver-call, and that genuinely helps when calls are expensive — but the benefit is fragile (it needs the trust region) and modest (cheap baselines capture most of it).

The honest reading: the real lever is the **cost signal** g — having *any* way to rank candidates by predicted quality — not the latent geometry specifically. Notice that "sample + screen(g)," which just uses g as a filter without any gradient steps, did as well as the sophisticated steering. The expensive, clever part (gradient descent in latent space) did less work than the cheap part (a good cost predictor used as a ranker).

This is *useful* news, not a defeat. It tells you the defensible version of the project is not "latent optimization is great." It is: "here is exactly where steering beats sampling-and-screening and retrieval — namely under a tiny solver budget — and here is the trust-region machinery that keeps it from self-destructing." The next experiment that would matter most is to make the problem harder (higher dimension, shifted test distribution) where the cheap baselines stop being so dominant, and see whether steering's small edge grows into a real one.

Glossary of the symbols

Symbol	Meaning
x	a position / candidate seed (here, 2 numbers)
$f(x;\theta)$	the landscape (cost) we minimize
θ	the problem-instance parameter (2 numbers); changes which valley is deepest
$c_k,\ d_k(\theta),\ \sigma$	well centers, depths, width
∇	gradient — direction of steepest increase
η	step size (learning rate)
B	budget = number of expensive solver calls allowed
$f^\star(\theta)$	true global minimum (from a dense grid)
z	latent code (4 numbers) — the searchable coordinate
$\mathrm{decode}(z,\theta)$	turns a code into a seed
$g(z,\theta)$	cost-to-go surrogate — cheap prediction of final cost
$\lambda\|z\|^2$	trust region — penalty for leaving the trustworthy region